arXiv:2602.01219v6 [cs.LG] 27 Sep 2026

Mixture-of-Top-k Attention: Efficient Attention via Scalable Fast Weights

Qishuai Wen, Zhiyuan Huang, Wei He, Xianghan Meng, and Chun-Guang Li
Beijing University of Posts and Telecommunications
{wqs, huangzhiyuan, mengxianghan, wei.he, lichunguang}@bupt.edu.cn

Abstract

The vanilla self-attention mechanism in Transformers can be viewed as a two-layer fast-weight MLP, whose weights are dynamically induced by inputs and whose hidden dimension is equal to the sequence length N. As the context extends, the expressive capacity of such an N-width MLP increases, but it becomes unscalable for extremely long sequences. Recently, this fast-weight perspective has motivated the Mixture-of-Experts (MoE) attention mechanism, which partitions the sequence into rigid blocks, treats them as fast-weight experts, and sparsely routes the tokens to them. In this paper, we elevate this perspective to a unifying framework for efficient attention mechanisms, interpreting them as making fast weights scalable through either routing or compression, and organizing them into a five-dimensional taxonomy. Then, we propose **Mi**xture-of-**T**op-k **A**ttention (**MiTA**), which employs a small set of landmark queries to gather top-k attended key-value pairs as query-aware and deformable routed experts, while compressing the N-width MLP into a narrower shared expert. Consequently, our MiTA improves the flexibility of prior MoE attention from rigid to deformable fast-weight experts, as well as the scalability of prior top-k attention from query-specific set to reusable top-k set. We conduct extensive experiments on vision tasks showing the superior effectiveness and efficiency of our MiTA, and also uncovering intriguing properties such as an emergent token-pruning effect and easy generalization from standard attention. Code is available at https://github.com/QishuaiWen/MiTA.

1 Introduction

Attention is the core operation of Transformers, which underpins today's success and wide application of deep learning. Intuitively, attention learns to store the context as key-value associations and retrieve this short-term memory via queries [1]. However, such an all-to-all lookup paradigm incurs quadratic computation and memory complexity in the sequence length, thereby hindering its scaling to long sequences. To this end, a plethora of efficient attention methods have been explored, albeit still lacking a unifying principle [2].

In recent years, several lines of work have converged on a promising viewpoint: the parameters of a two-layer MLP can be viewed as key-value pairs [3], and the key-value pairs in full attention can be viewed as the fast (i.e., input-dependent) weights of an MLP [4, 5]. Therefore, pursuing an efficient attention can be framed as a *fast-weight scaling* problem, and draw inspiration from slow-weight (i.e., parameter) scaling approaches, such as weight tying [6], model pruning [7], and conditional computation [8].

Recent work along this direction has demonstrated that sparse attention inspired by Mixture-of-Experts (MoE) can scale up the effective sequence length, deliver wall-clock speedups, and accommodate large-scale pretraining [9, 10]. In contrast to traditional MoE, whose experts are static and structured model parameters (i.e., slow weights), their experts are input-dependent fast weights [11],

Preprint.

Table 1: A five-dimensional taxonomy for efficient attention methods from the fast-weight scaling perspective. We present expert type and expert count together in the table. A detailed analysis of this taxonomy and how MiTA fits into it can be found in Appendix B.

Method	Scaling strategy	Expert type and count	Expert construction	Routing topology
Linear Attention [2020] MHLA [13]	Compression	One shared linear layer m linear layers	Kernelization Local prior & kernelization	All-to-one m times $\frac{N}{m}$-to-one
Linformer [14] PVT [15]	Compression	One shared MLP	Learnable projection	All-to-one
Agent Attention [16]	Compression	One shared MLP	Landmark probing	All-to-one
TTT [17] ViT^3 [5]	Compression	One shared module	Test-time training	All-to-one
Deformable DETR [2021] DAT [19]	Routing	N MLPs One shared MLP	Offset predicting	N times one-to-one N-to-one
BRA [20] MoBA [9] NSA [10]	Routing Routing Compression & Routing	$\frac{N}{n}$ MLPs	Locality prior	$\frac{N}{n}$-to-$\frac{N}{n}$ N-to-$\frac{N}{n}$
Spark Attention [21] DSA [22]	Routing	N MLPs	Light-weight skim	N times one-to-one
MiTA (ours)	Compression & Routing	m MLPs	Landmark probing	N-to-m

more precisely, subsets of key-value pairs. Routing queries to the fast-weight experts sparsely, MoE attention reduces the complexity from quadratic to linear in sequence length N.

Roughly speaking, the central challenge in scaling the fast weights via MoE is to construct fast-weight experts from unstructured key-value pairs. For example, given the spatial and temporal locality priors in many modalities, a straightforward, hardware-friendly, albeit naive, approach is to partition the sequence into contiguous, non-overlapping, fixed-size blocks, and regard these blocks as fast-weight experts (i.e., a mixture of blocks or chunks) [9, 10, 23, 24]. After that, routing vectors are obtained by aggregating each block into a single vector, e.g., via average pooling or parameterized modules.

Since splitting the N-width fast-weight MLP into a mixture of such rigid blocks is coarse and suboptimal, subsequent work [25] has sought to improve upon the splitting scheme. Notably, we argue that top-k attention [21, 22] can be interpreted as spawning N k-width sub-MLPs from the N-width MLP, via gathering top-k key-value pairs attended by each query. Therefore, the rigid (fixed-shape) expert in prior MoE attention methods are replaced by top-k attention with top-k key-value pairs. However, instead of the width of the fast-weight MLP in full attention, it is the number of such sets of top-k key-value pairs that always equals N in their top-k attention, which still limits its scalability.

On the other hand, unlike the methods mentioned above that scale fast weights by accessing a subset of them, linear attention [12] and Test-Time Training (TTT) [17] compress the N-width MLP into one (or several [13]) light-weight module(s), analogous to model compression [26] and knowledge distillation [27]. Therefore, from a fast-weight scaling perspective, we divide the scaling approaches of existing efficient attention methods discussed in this paper into two general categories: a) scaling by routing, and b) scaling by compression (see Fig. 1 for an illustration). Scaling by compression alone sacrifices a precise access to the original key-value pairs; whereas scaling by routing alone lacks a global summary of the full context. Although these two approaches are not mutually exclusive, most existing methods typically adopt only one of them.

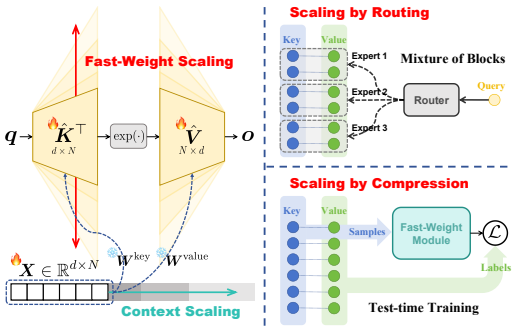


Figure 1: As the context extends, the width of the two-layer fast-weight MLP induced by full attention increases accordingly. We categorize efficient fast-weight scaling approaches into two strategies: a) scaling by routing and b) scaling by compression, and illustrate each with a representative method: MoBA [9] and TTT [17].

In this paper, we first elevate the fast-weight scaling perspective into a five-dimensional taxonomy that accommodates a broad spectrum of prior efficient attention methods (see Tab. 1). While such a taxonomy is by no means complete, it paves the way towards a unified framework and serves as a

useful design principle. Then, we propose **Mi**xture-of-**T**op-k **A**ttention (**MiTA**), which employs a small set of landmark queries to gather top-k attended key-value pairs as query-aware and deformable routed experts, while compressing the N-width MLP into a narrower shared expert. Consequently, our MiTA improves the flexibility of prior MoE attention from rigid to query-aware and deformable experts, and the scalability of prior top-k attention from N private (i.e., query-specific) to a tunable number m of reusable top-k sets; and b) combines the two scaling strategies, i.e., , routing and compression, together, thereby addressing the limitations identified within the taxonomy.

Specifically, our MiTA queries original key-value pairs using landmark queries to obtain corresponding landmark values. The resulting landmark query-value pairs (i.e., compressed key-value pairs) form the shared expert, where landmark queries act as keys and landmark values as values. Meanwhile, MiTA reorganizes the original key-value pairs into routed experts by gathering the top-k key-value pairs attended by each landmark query. As shown in Fig. 2, for each query, MiTA concatenates these compressed key-value pairs with a routed subset of the original key-value pairs.

Contributions. The contributions of this paper are three-fold:

1. **Conceptually**, we introduce a unified perspective that views efficient attention mechanisms as scaling fast weights, systematically organizing existing methods into a five-dimensional taxonomy and naturally revealing their limitations: the rigidity of experts, the non-reusability of top-k sets, and the need to combine routing and compression.
2. **Technically**, we propose a novel efficient attention mechanism, MiTA, which constructs a tunable number of query-aware, deformable expert (equivalently, reusable and routable top-k sets) and achieves scalable fast weights through both routing and compression, thereby addressing these limitations in a principled manner.
3. **Experimentally**, we validate the effectiveness and efficiency of MiTA on vision tasks, demonstrating that our MiTA surpasses strong baselines by a large margin while using less wall-clock time, efficiently approaching the expressive capacity of full attention to a remarkable extent.

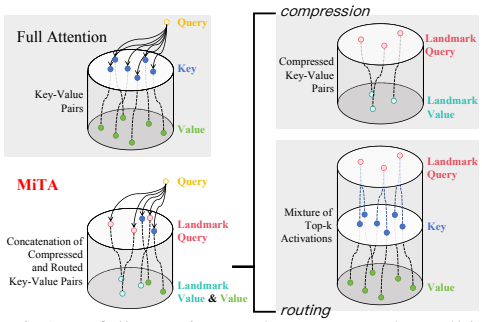


Figure 2: Illustration for our MiTA. In full attention, each query attends to all key-value pairs. In our MiTA, it attends to the concatenation of a small number (i.e., , 3 as shown above) of the compressed key-value pairs and a routed subset of the original key-value pairs (i.e., , 2 as shown above).

2 Related Work

2.1 Fast-Weight Scaling

In contrast to slow weights (i.e., model parameters that remain unchanged after training and encode persistent knowledge), fast weights are input-conditioned and act as temporary parameters. They can be viewed as short-term memory [28], playing an important role in meta-learning [29]. While most scaling efforts have focused on expanding slow weights (e.g., increasing model width and depth) [30, 31], extending Transformers' sequence length turns out to implicitly scale fast weights.

For instance, Test-Time Training (TTT) [17] effectively compresses the N-width two-layer fast-weight MLP in full attention into a smaller fast-weight module, likewise for linear attention [12] into a linear layer [5] and Linformer [14] into a narrower MLP. Rather than scaling by compressing the full fast-weight MLP, MoE attention factorizes it into experts and access them sparsely via routing.

Our MiTA combines the two strategies mentioned above. It compresses full key-value pairs into a smaller set (a narrower MLP), which offers a coarse yet global summary, and then leverages the top-k activations of the landmark queries to identify underlying experts, which enables precise retrieval.

2.2 MoE-Inspired Sparse Attention

Previous sparse attention methods typically focus on the design of fixed sparse patterns (e.g., the local window, axial stripe [32], and the vertical-slash pattern [33]) or learnable ones (e.g., hash buckets [34] and k-means clusters [35].) To maintain a global connectivity, shared memory (i.e., tokens that attend to and are attended by all tokens) is also introduced [36]. However, these patterns are either task-dependent [37] or too burdensome to achieve wall-clock speedup [38].

Recently, sparse attention inspired by Mixture-of-Experts (MoE) has been applied to pre-training large language models, yielding extended effective context with reduced overhead [9, 10]. Its key advantage is the routing mechanism, which enables query-aware selection within the key-value cache. Consequently, despite coarse selection granularity (e.g., evenly split, non-overlapping blocks), the sparse pattern is highly flexible and can vary across queries.

In this paper, we argue that, beyond routing, the experts can also be query-aware and deformable. Specifically, for an arbitrarily long sequence, we can build a fixed, yet configurable number of experts by gathering semantically related key-value pairs. A similarly motivated line of research is to replace the fixed, non-overlapping rectangular patchification in ViTs [39] with a deformable, content-adaptive tokenization scheme [40].

2.3 Deformable Attention

More broadly, conditional computation, the idea underlying MoE, has been explored through the deformable convolution [41] in convolutional neural networks. And deformable attention was introduced for object detection by Zhu et al. [18] and later generalized to Vision Transformers by Xia et al. [19]. Given a query, while being blind to keys, deformable attention predicts offsets relative to a small set of default positions of the keys, thereby inducing a query-aware, deformable sparse attention pattern (i.e., a fast-weight expert in our context).

Recent top-k attention methods, e.g., Spark Attention [21] and DeepSeek Sparse Attention [22], which have been applied to the training of Gemma 3n and DeepSeek-V3.2, respectively, can also be viewed as improved deformable attention. Instead of predicting the spatial positions solely from the query as in prior deformable attention, these methods locate the top-k key-value pairs by allowing each query to take a lightweight skim of the full keys, e.g., using partial features [21] or low precision (FP8) [22].

Our MiTA retains the key advantage of top-k attention by constructing deformable experts (i.e., sparse patterns) that depend on both queries and keys. However, unlike the methods discussed above, we replace the per-query offset predicting [18, 19] or light-weight skim [21, 22] with more efficient per-query routing to reusable sparse patterns, thereby further improving scalability.

3 Methods

3.1 Efficient Attention as Scaling Fast Weights

This subsection will first present the mathematical formulation of the fast-weight MLP equivalent to full attention, and then reinterpret efficient attention as a fast-weight scaling problem. From this perspective, a taxonomy of previous efficient attention methods in terms of how they scale fast weights will be given.

Fast-weight MLP. Formally, full attention (i.e., scaled dot-product attention [42]) can be written as:

$$\mathrm{Atten}(\boldsymbol{q}, \boldsymbol{K}, \boldsymbol{V}) = \boldsymbol{V}\,\mathrm{softmax}\left(\boldsymbol{K}^\top \boldsymbol{q}/\sqrt{d}\right), \tag{1}$$

where $\boldsymbol{q} \in \mathbb{R}^d$ is a query, and the columns of $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{d\times N}$ are the keys and values, respectively. By contrast, a D-width two-layer MLP, i.e., the feed-forward network (FFN) in Transformers, is:

$$\mathrm{MLP}_\sigma(\boldsymbol{x} \mid \boldsymbol{W}_1; \boldsymbol{W}_2) = \boldsymbol{W}_2\sigma\left(\boldsymbol{W}_1^\top \boldsymbol{x} + \boldsymbol{b}_1\right) + \boldsymbol{b}_2, \tag{2}$$

where $\boldsymbol{x} \in \mathbb{R}^d$ is the input vector, $\boldsymbol{W}_1, \boldsymbol{W}_2 \in \mathbb{R}^{d\times D}$ are weights, $\boldsymbol{b}_1 \in \mathbb{R}^D$ and $\boldsymbol{b}_2 \in \mathbb{R}^d$ are biases[1], and $\sigma(\cdot)$ is an element-wise activation function (e.g., ReLU). Note that full attention in Eq. (1) is

[1] For notion simplicity, biases are omitted from the argument.

equivalent to the following MLP:

$$\mathrm{MLP}_{\exp}(\boldsymbol{q} \mid \hat{\boldsymbol{K}}; \hat{\boldsymbol{V}}) = \hat{\boldsymbol{V}} \exp\left(\hat{\boldsymbol{K}}^\top \boldsymbol{q}\right), \tag{3}$$

$$\hat{\boldsymbol{K}} = \boldsymbol{K}/\sqrt{d},\ \hat{\boldsymbol{V}} = \boldsymbol{V}/\left(\exp(\hat{\boldsymbol{K}}^\top \boldsymbol{q})^\top \mathbf{1}_N\right), \tag{4}$$

where the weights are given by the scaled keys and values, the biases are all zero, and the activation function is the exponential function $\exp(\cdot)$. Therefore, we have demonstrated that full attention in Eq. (1) is equivalent to the *N-width, fast-weight, two-layer MLP* in Eq. (3).

Unbounded fast-weight scaling. Because the hidden dimension of the fast-weight MLP in Eq. (3) always equals to the sequence length N, the per-query overhead grows linearly with N rather than fixed as in the slow-weight MLP in Eq. (2). Thus, under the all-to-all lookup paradigm, processing N queries incurs an overall $\mathcal{O}(N^2)$ overhead. This unbounded, rapid growth prevents scaling the fast weights to arbitrarily long sequences. While this challenge is commonly described as the quadratic complexity of full attention in the literature on efficient attention, we refer to it as the *unbounded fast-weight scaling* issue.

Fast-weight scaling taxonomy. Although we have discussed relevant efficient-attention methods in Sec. 2, they have not been systematically organized into a unified fast-weight scaling perspective. We therefore propose a five-dimensional taxonomy: a) *scaling strategy*, i.e., scaling the N-width fast-weight MLP either by compressing it into a light-weight module or by routing each query to a subset of it; b) *expert count*, i.e., how many fast-weight experts are constructed (in particular, scaling by compression typically yields a single shared expert); c) *expert type*, i.e., what module each expert takes (e.g., an MLP or a linear layer); d) *expert construction*, i.e., how experts are formed from the key-value pairs; and e) *routing topology*, i.e., the query-expert assignment pattern. The mapping of existing methods to this taxonomy is summarized in Tab. 1. We defer the in-depth discussion until Appendix B.

3.2 Mixture-of-Top-k Attention (MiTA)

Motivated by the limitations identified through our fast-weight scaling perspective, our method has two design goals: a) moving from rigid to query-aware, deformable experts, and from private (i.e., query-specific) to reusable top-k key-value pairs, and b) combining compression and routing. We begin with the second goal, which is simpler, and show that it naturally leads to the first, which is more important.

Note that both scaling by compression and scaling by routing have the inherent limitations. Compressing the full fast-weight MLP inevitably discards information, although the information loss can be mitigated by adopting milder compression schemes (e.g., piecewise compression [13]) or by compressing into a more expressive module (e.g., via test-time training [17]). In contrast, scaling by routing is faithful and lossless, but it often lacks a global view of the context.

To combine the strengths of both strategies, i.e., to retain a compact global summary while enabling precise, token-level retrieval, we introduce a small set of *landmark queries* $\tilde{\boldsymbol{Q}} \in \mathbb{R}^{d\times m}$ with $m \ll N$. These landmark queries probe the full key-value cache and then, via cross-attention, compress it into a global fast-weight module, while simultaneously forming deformable fast-weight experts by gathering the top-k activated (i.e., attended) key–value pairs for each landmark query.

This design is motivated by two observations: a) register tokens can attend to distinct semantic regions of an image [43]; and b) class embeddings [44] (or object queries [45] and mask embeddings [46]) provide a compact global summary for dense prediction tasks [47] and even image generation [48]. The landmark queries can be obtained in various ways, e.g., by assigning a set of

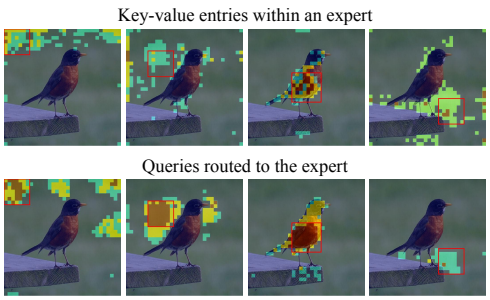


Figure 3: Visualization of experts' gathered key-value pairs, and routed queries. The red box marks the local window from which the landmark query is obtained via average pooling. The attention heatmap (averaged over attention heads) indicates key-value pairs within each expert (top row) and the queries routed to it (bottom row). Notably, neither the expert's key–value pairs nor the routed queries are confined to the local window.

learnable slow weights or by downsampling the sequence using a convolutional module. In this work, we simply apply average pooling over uniformly spaced, equal-sized windows, as shown in Fig. 3.

Specifically, the i-th landmark query $\tilde{\boldsymbol{q}}_i$ (among m in total) defines the i-th expert $\mathcal{E}_i$ as follows:

$$\mathcal{E}_i(\boldsymbol{q}) = \mathrm{Atten}(\boldsymbol{q}, \boldsymbol{K}^{(i)}, \boldsymbol{V}^{(i)}),\ i \in \{1, \dots, m\}, \tag{5}$$

$$\boldsymbol{K}^{(i)} = \boldsymbol{K}_{:,\mathcal{I}_i},\ \boldsymbol{V}^{(i)} = \boldsymbol{V}_{:,\mathcal{I}_i} \in \mathbb{R}^{d\times k}, \tag{6}$$

$$\mathcal{I}_i = \mathrm{Top}_k\left(\boldsymbol{K}^\top \tilde{\boldsymbol{q}}_i\right) \in \{1, \dots, N\}^k, \tag{7}$$

where $\boldsymbol{q}$ is a query routed to expert $\mathcal{E}_i$, and $\boldsymbol{K}^{(i)}, \boldsymbol{V}^{(i)}$ are the top-k key-value pairs activated by $\tilde{\boldsymbol{q}}_i$. This approach effectively reorganizes the full key-value pairs into a mixture of top-k key-value pairs (i.e., a mixture of deformable experts). We directly use $\tilde{\boldsymbol{q}}_i$ as the routing vector for expert $\mathcal{E}_i$. Then the routing logits from the N queries $\boldsymbol{Q} \in \mathbb{R}^{d\times N}$ to the m experts is $\boldsymbol{Q}^\top \tilde{\boldsymbol{Q}} \in \mathbb{R}^{N\times m}$. We denote the index of the rank-j expert that a query $\boldsymbol{q}$ is routed to under these logits as $e_j(\boldsymbol{q}) \in \{1, \dots, m\}$.

Moreover, regarding the compressed global module, which can be viewed as a shared expert [49], we extract a set of landmark values $\tilde{\boldsymbol{V}} \in \mathbb{R}^{d\times m}$ via cross-attention using the landmark queries. The i-th landmark value corresponding to the i-th landmark query is given by:

$$\tilde{\boldsymbol{v}}_i = \mathrm{Atten}(\tilde{\boldsymbol{q}}_i, \boldsymbol{K}, \boldsymbol{V}). \tag{8}$$

Then, the shared expert $\tilde{\mathcal{E}}$ is defined as:

$$\tilde{\mathcal{E}}(\boldsymbol{q}) = \mathrm{Atten}(\boldsymbol{q}, \tilde{\boldsymbol{Q}}, \tilde{\boldsymbol{V}}), \tag{9}$$

where landmark queries act as the keys in standard attention. Note that the computations in Eqs. (7) and (8) can be largely shared, where landmark queries are both querying the keys. Likewise, Eq. (9) can reuse the routing logits. Therefore, the computations of the two scaling strategies are tightly coupled in our MiTA.

Rather than a straightforward MoE implementation, which isolates the computation within each expert and aggregates their outputs via a weighted sum, we concatenate the experts as a single standard attention, as shown in Fig. 2. Therefore, our **MiTA** can be written as:

$$\mathrm{MiTA}(\boldsymbol{q}) = \boldsymbol{V}^\star \,\mathrm{softmax}({\boldsymbol{K}^\star}^\top \boldsymbol{q}/\sqrt{d}), \tag{10}$$

$$\boldsymbol{K}^\star = [\tilde{\boldsymbol{Q}}, \boldsymbol{K}^{(e_1(\boldsymbol{q}))}, \dots, \boldsymbol{K}^{(e_s(\boldsymbol{q}))}], \tag{11}$$

$$\boldsymbol{V}^\star = [\tilde{\boldsymbol{V}}, \boldsymbol{V}^{(e_1(\boldsymbol{q}))}, \dots, \boldsymbol{V}^{(e_s(\boldsymbol{q}))}], \tag{12}$$

where $\boldsymbol{K}^\star, \boldsymbol{V}^\star \in \mathbb{R}^{d\times(m+ks)}$ are the key-value pairs that a query $\boldsymbol{q}$ attends to in our MiTA, and s is the number of routed experts (not counting the shared expert) per query. Note that, in practice, the attention can still be computed expert-wise and then combined, thanks to the online softmax [50], as in the FlashAttention [51] implementation of Lu et al. [9].

Implementations. MiTA is conceptually simple yet effective, but technically non-trivial to realize with actual wall-clock speedups. In particular, we implement our MiTA in Eq. (10) with $s = 1$ (as in Llama 4), i.e., each query is fed to the shared expert and routed to exactly one additional fast-weight expert. The pseudocode of our MiTA is given in Algorithm 1 (Appendix B) which matches queries to experts by sorting queries according to their expert assignments $e_1(\boldsymbol{q})$. The more general but more complex case with $s > 1$ can be implemented via MoBA [9] at the cost of reduced speed.

Complexity analysis. The computational complexity of Eq. (10) alone is $\mathcal{O}(N(m + ks))$, whereas full attention incurs $\mathcal{O}(N^2)$; in practice, $N \gg m + ks$. Moreover, the cross-attention operations in Eqs. (8) and (10) can be accelerated by the FlashAttention family [38, 51, 52]. The primary bottleneck comes from the gather operation in Eq. (6): it entails irregular (random) memory access and is therefore likely to make the overall pipeline memory-bound. Nonetheless, Deepseek Sparse Attention suggests that this limitation can be mitigated via its optimized implementation [22, Fig. 3]. Moreover, compared with prior top-k attention [21, 22], which instantiate a private fast-weight expert per query (N experts in total) without routing, our MiTA uses a fixed (yet tunable) number (i.e., m) of fast-weight experts, which is more hardware-friendly, at the cost of an N-to-m routing step, which is explored in MoBA [9]. In summary, the hardware operations required by MiTA have been validated by prior engineering-centric work, allowing us to focus on the fast-weight scaling perspective, its taxonomy, the construction of a tunable number of query-aware, deformable fast-weight experts, and the combination of compression and routing.

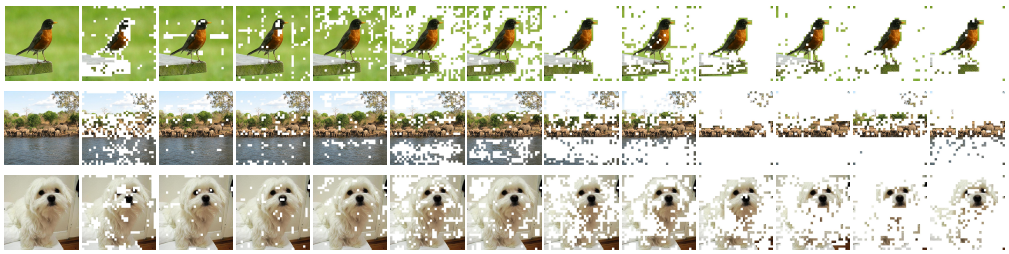


Figure 4: The token pruning effect of MiTA. Each row visualizes, for each layer, the positions of key-value pairs (aggregated over heads) selected as experts; the leftmost image shows the original input. In later layers, most tokens are effectively "pruned" (i.e., not selected as experts), and attention concentrates on class-relevant regions. The examples are sampled from the ImageNet-1K training set.

4 Experiments

To verify the efficacy of our MiTA, we conduct image classification experiments on ImageNet-1K [53] and semantic segmentation experiments on ADE20K [54]. Moreover, we assess the long sequence modeling capability of MiTA on the Long Range Arena benchmark [55], and report its wall-clock training/inference speed against full attention on extremely long sequences. A detailed ablation study is provided in Appendix A. Furthermore, we study the robustness (or generalization) of MiTA under varying expert count (m) and width (k) with a fixed trained model, and examine the broader algorithmic generalization of Transformers across attention mechanisms; see Appendix C for details.

Table 2: Results on ImageNet-1K training from scratch. ^{Bias}: using the agent bias [16]; ^{DWC}: using depth-wise convolution; ^{Gate}: adding the attention output to the residual with data-dependent gating. PolaFormer [56], MHLA [13], and FLatten [57] are variants of linear attention. BRA [20] and Agent Attention [16] can be viewed as degenerate cases of MiTA, scaling by only routing and only compression, respectively.

Methods	# Params	FLOPs	Acc. (%)
DeiT-T	5.7M	1.2G	72.2
Linear-DeiT-T	5.7M	1.1G	68.0
PolaFormer-DeiT-T	5.7M	1.2G	68.1
MHLA-DeiT-T	5.7M	1.1G	69.2
FLatten-DeiT-T	6.1M	1.1G	70.2
BRA-DeiT-T	5.7M	1.1G	70.2
Agent-DeiT-T	5.7M	1.1G	70.3
Agent-DeiT-T^{Bias}	6.0M	1.2G	71.1
MiTA-DeiT-T	5.7M	1.1G	71.1 (-1.1)
MiTA-DeiT-T^{DWC}	5.7M	1.1G	73.4 (+1.2)
DeiT-S	22M	4.6G	79.9
Agent-DeiT-S	22M	4.4G	78.5
Agent-DeiT-S^{Bias}	23M	4.4G	79.1
MiTA-DeiT-S	22M	4.4G	79.8 (-0.1)
MiTA-DeiT-S^{DWC}	22M	4.4G	80.6 (+0.7)
MiTA-DeiT-S$^{\text{DWC,Gate}}$	22M	4.7G	81.2 (+1.3)

Table 3: Comparison with SOTA and efficient ViTs on ImageNet-1K. We compare against a recent state-of-the-art ViT variant, ViT-5 [58], as well as strong baselines such as DeiT and DeiT-III [59]. We also include efficient ViTs, including sparse models (SViTE [60] and Sparsifiner [61]) and scaling-by-compression models (InLine [62] and Agent Attention).

Methods	# Params	FLOPs	Acc. (%)
ViT-5-S	22M	4.7G	82.2
DeiT-S	22M	4.6G	79.9
DeiT-III-S	22M	4.6G	81.4
SViTE-S	11M	2.5G	79.7
Sparsifiner-S	23M	4.5G	79.9
InLine-DeiT-S	17M	5.0G	80.2
Agent-DeiT-S$^{\text{Bias,DWC}}$	23M	4.4G	80.5
MHLA-DeiT-S^{DWC}	22M	4.2G	81.0
MiTA-ViT-5-S	22M	4.5G	81.3 (-0.9)
MiTA-ViT-5-S^{DWC}	22M	4.5G	81.7 (-0.5)

Table 4: Results on ADE20K semantic segmentation. $^{\triangledown}$: the attention mechanism is directly replaced, rather than being natively pretrained. Mask Transformer is the decoder introduced in the pioneering work Segmenter [44] for Transformer-based segmentation, while DEPICT [47] and CBT [63] are two principled improvements upon it.

Backbones	Seg. head	Resolution	FLOPs	mIoU (%)
ViT-T	Mask Trans.	512^2	13G	38.1
ViT-T	DEPICT	512^2	13G	39.3
ViT-T	CBT	512^2	12G	39.1
MiTA-ViT-T$^{\triangledown}$	CBT	512^2	7G (↓42%)	36.5 (-2.6)
ViT-S	Mask Trans.	512^2	38G	45.3
ViT-S	DEPICT	512^2	35G	46.7
ViT-S	CBT	512^2	33G	45.8
MiTA-ViT-S$^{\triangledown}$	CBT	512^2	25G (↓24%)	44.5 (-1.3)
ViT-B	Mask Trans.	512^2	128G	48.5
ViT-B	DEPICT	512^2	111G	49.2
ViT-B	CBT	512^2	111G	49.3
MiTA-ViT-B$^{\triangledown}$	CBT	512^2	95G (↓14%)	48.1 (-1.2)
ViT-L	Mask Trans.	640^2	667G	51.8
ViT-L	DEPICT	640^2	628G	52.9
ViT-L	CBT	640^2	622G	53.3
MiTA-ViT-L$^{\triangledown}$	CBT	640^2	508G (↓18%)	50.5 (-2.8)

Baselines. Note that not all methods listed in Tab. 1 are included in our experiments for the following reasons: (i) although representative, many have become outdated; and (ii) some are designed specifically for language tasks and are therefore not directly comparable in our vision setting. Nonetheless, we include comparisons with their vision counterparts that are based on similar ideas.

4.1 Image Classification

Current advances in efficient attention on ImageNet-1K are typically accompanied by extra components such as depth-wise convolutions, which obscure the expressiveness gap between the proposed efficient mechanisms and full attention. To this end, we report a fair comparison in Tab. 2. Specifically, we train all models using the DeiT [64] recipe and vary only the attention mechanism. Meanwhile, in Tab. 3, we compare our best-performing ViT variant based on MiTA against state-of-the-art and efficient ViTs.

Implementation details. We use an image size of 224 with a patch size of 16, and set $s = 1$, $m = k = 25$ for MiTA.[2] Under this setting, each query attends to $m + ks = 50$ key-value pairs.

Results. As indicated in Tab. 2, without extra components, MiTA outperforms other efficient attention by a substantial margin (at least 0.8% and up to 3.1%). Although Agent Attention can narrow the gap by using agent bias [3], it still underperforms MiTA on small-sized models by 0.7%. Moreover, a severe limitation of this trick is that it is currently not supported by FlashAttention. As shown in Tab. 3, when equipped with the architectural modifications explored in ViT-5 [58], MiTA outperforms a range of strong ViT baselines and approaches state-of-the-art performance within a small margin while using fewer FLOPs.

4.2 Semantic Segmentation

ADE20K [54] is a scene-centric dataset for semantic segmentation, which requires higher image resolutions to preserve fine-grained details such as object boundaries. In Tab. 4, we apply MiTA to the backbone for efficient visual encoding and compare against methods with ViT backbones and Transformer-based segmentation heads.

[2]We set $s = 1$ throughout this work for a preliminary study. As analyzed in Sec. B, this choice does not fully exploit the expressiveness of MiTA.

[3]An agent bias $\boldsymbol{B} \in \mathbb{R}^{N\times m}$ is added to attention scores as $\mathrm{softmax}(\boldsymbol{K}^\top \tilde{\boldsymbol{Q}} + \boldsymbol{B})$.

Table 5: Results on LRA benchmark. For each task, we report the accuracy and training throughput (steps/s), along with the average accuracy across tasks and the total training wall-clock time (hours). ‡: a route-only variant (i.e., the compression or local window branch is removed), where the number of entries attended per query kept unchanged by increasing k.

Methods	ListOps (2K)	Text (4K)	Retrieval (4K)	Image (1K)	Pathfinder (1K)	Avg. / Tot.
Standard Atten	37.35 / 12.7	63.63 / 3.8	79.90 / 1.9	40.38 / 5.7	69.68 / 5.7	58.19 / 10.6
Reformer	18.55 / 34.1	65.08 / 16.8	78.77 / 8.5	44.04 / 8.3	69.04 / 8.5	55.10 / 4.5
Linformer	38.05 / 35.0	56.69 / 24.6	78.41 / 13.7	39.56 / 11.9	67.13 / 11.9	55.97 / 3.1
Performer	17.84 / 28.0	65.13 / 16.5	77.18 / 8.9	36.64 / 8.1	70.20 / 8.0	53.40 / 4.7
Nyströmformer	29.69 / 22.0	65.84 / 16.7	79.68 / 8.8	36.83 / 4.5	72.50 / 4.5	56.91 / 7.3
Agent Atten	36.59 / 40.1	63.86 / 38.9	79.98 / 24.3	38.92 / 19.5	71.77 / 19.1	58.22 / 1.9
MoBA‡	37.10 / 30.4	62.35 / 15.8	76.64 / 8.2	37.92 / 8.6	69.67 / 8.4	56.74 / 4.6
MiTA‡	37.20 / 25.4	64.09 / 22.2	79.92 / 13.1	40.17 / 5.3	72.66 / 6.3	58.81 / 5.5
MiTA	37.20 / 52.5 (×4.1)	63.47 / 37.7 (×9.9)	79.88 / 19.6 (×10.3)	40.72 / 13.6 (×2.4)	73.26 / 15.3 (×2.7)	58.91 / 2.4 (↓77%)

Implementation details. ViT backbones [65] are pretrained on ImageNet-21K with an image size of 384 and a patch size of 16. We use a consistent patch size of 16 when training on ADE20K, and set $s = 1$, $m = k = 49$ for MiTA. Under this setting, each query attends to $m + ks = 98$ key-value pairs, while it is 1,024 (for 512^2 image resolution) or 1,600 (640^2 resolution) in full attention.

Results. As depicted in Tab. 4, MiTA remarkably reduces the FLOPs (by up to 42%) while achieving comparable segmentation performance. Note that MiTA is not fully exploited here because the backbone is not natively pretrained with it.

4.3 Evaluation on Long Sequences

As a preliminary validation for long-sequence modeling, we evaluate MiTA on the Long Range Arena (LRA) benchmark [55] and compare against several classical efficient attention methods [34, 14, 66, 67]. We report their accuracy and training throughput (and wall-clock time) on LRA in Tab. 5, while the inference throughput improvement of MiTA over standard attention is shown in Fig. 5.

Implementation details. The training configuration follows Xiong et al. [67] and hyperparameters are kept consistent across all methods for a fair comparison. Thus, the reported accuracies are not tuned to fully exploit the best performance of each approach. We set $s = 1$ and $m = k = 128$ for MiTA. Throughput and wall-clock time reported in Tab. 5 and Fig. 5 are measured on a 24GB RTX 4090. Standard attention is implemented with PyTorch's fused kernel (i.e., FlashAttention). More specifically, results in Fig. 5 are measured on a three-layer Transformer with an embedding dimension of 128, and the batch size is tuned for each run to maximize throughput.

Results. As shown in Tab. 5, MiTA achieves accuracy comparable to standard attention, while providing a substantial speedup that reduces total training time by 77%. Notably, the route-only variant of MiTA also performs well but is slower, since gathering more key-value pairs incurs higher overhead. This suggest that using a shared, compressed set of key-value pairs can effectively represent thousands of routed original pairs.

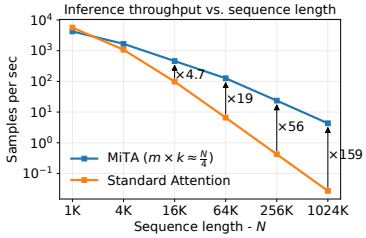


Figure 5: Inference throughput.

5 Conclusion

We adopted fast-weight scaling as a unifying perspective for efficient attention and introduced a five-dimensional taxonomy to identify the limitations of prior methods. Moreover, we proposed a novel efficient attention method, termed Mixture-of-Top-k Attention (MiTA), which constructs a tunable number of query-aware and deformable fast-weight experts and bridges the two different scaling strategies together, thereby addressing their inherent limitations. We provided a detailed analysis of both the taxonomy and our MiTA, suggesting a principled and promising path for developing efficient attention from the fast-weight scaling perspective.

References

[1] Alberto Bietti, Vivien Cabannes, Diane Bouchacourt, Hervé Jégou, and Léon Bottou. Birth of a transformer: A memory viewpoint. In *NeurIPS*, 2023.

[2] Jintao Zhang, Rundong Su, Chunyu Liu, Jia Wei, Ziteng Wang, Pengle Zhang, Haoxu Wang, Huiqiang Jiang, Haofeng Huang, Chendong Xiang, Haocheng Xi, Shuo Yang, Xingyang Li, Yuezhou Hu, Tianyu Fu, Tianchen Zhao, Yicheng Zhang, Youhe Jiang, Chang Chen, Kai Jiang, Huayu Chen, Min Zhao, Xiaoming Xu, Jun Zhu, and Jianfei Chen. A survey of efficient attention methods: Hardware-efficient, sparse, compact, and linear attention. 2025.

[3] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *EMNLP*, 2021.

[4] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, 2021.

[5] Dongchen Han, Yining Li, Tianyu Li, Zixuan Cao, Ziming Wang, Jun Song, Yu Cheng, Bo Zheng, and Gao Huang. ViT^3: Unlocking test-time training in vision. *arXiv preprint arXiv:2512.01643*, 2025.

[6] Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, et al. Parallel loop transformer for efficient test-time computation scaling. *arXiv preprint arXiv:2510.24824*, 2025.

[7] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *TPAMI*, 46(12):10558–10578, 2024.

[8] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. In *NeurIPS*, 2021.

[9] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. MoBA: Mixture of block attention for long-context llms. *arXiv preprint arXiv:2502.13189*, 2025.

[10] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *ACL*, 2025.

[11] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

[12] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020.

[13] Kewei Zhang, Ye Huang, Yufan Deng, Jincheng Yu, Junsong Chen, Huan Ling, Enze Xie, and Daquan Zhou. MHLA: Restoring expressivity of linear attention via token-level multi-head. *arXiv preprint arXiv:2601.07832*, 2026.

[14] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[15] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *ICCV*, 2021.

[16] Dongchen Han, Tianzhu Ye, Yizeng Han, Zhuofan Xia, Siyuan Pan, Pengfei Wan, Shiji Song, and Gao Huang. Agent attention: On the integration of softmax and linear attention. In *ECCV*, 2024.

[17] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *ICML*, 2025.

[18] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: Deformable transformers for end-to-end object detection. In *ICLR*, 2021.

[19] Zhuofan Xia, Xuran Pan, Shiji Song, Li Erran Li, and Gao Huang. Vision transformer with deformable attention. In *CVPR*, 2022.

[20] Lei Zhu, Xinjiang Wang, Zhanghan Ke, Wayne Zhang, and Rynson W. H. Lau. BiFormer: Vision transformer with bi-level routing attention. In *CVPR*, 2023.

[21] Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J Willcock, et al. Spark transformer: Reactivating sparsity in FFN and attention. In *NeurIPS*, 2025.

[22] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. DeepSeek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[23] Jianzong Wu, Liang Hou, Haotian Yang, Xin Tao, Ye Tian, Pengfei Wan, Di Zhang, and Yunhai Tong. VMoBA: Mixture-of-block attention for video diffusion models. *arXiv preprint arXiv:2506.23858*, 2025.

[24] Shengqu Cai, Ceyuan Yang, Lvmin Zhang, Yuwei Guo, Junfei Xiao, Ziyan Yang, Yinghao Xu, Zhenheng Yang, Alan Yuille, Leonidas Guibas, et al. Mixture of contexts for long video generation. *arXiv preprint arXiv:2508.21058*, 2025.

[25] Weinan Jia, Yuning Lu, Mengqi Huang, Hualiang Wang, Binyuan Huang, Nan Chen, Mu Liu, Jidong Jiang, and Zhendong Mao. MoGA: Mixture-of-groups attention for end-to-end long video generation. *arXiv preprint arXiv:2510.18692*, 2025.

[26] Jimmy Ba and Rich Caruana. Do deep nets really need to be deep? In *NeurIPS*, 2014.

[27] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[28] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

[29] Louis Kirsch, James Harrison, Jascha Sohl-Dickstein, and Luke Metz. General-purpose in-context learning by meta-learning transformers. *arXiv preprint arXiv:2212.04458*, 2022.

[30] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[31] Mingxing Tan and Quoc Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.

[32] Xiaoyi Dong, Jianmin Bao, Dongdong Chen, Weiming Zhang, Nenghai Yu, Lu Yuan, Dong Chen, and Baining Guo. CSWin transformer: A general vision transformer backbone with cross-shaped windows. In *CVPR*, 2022.

[33] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *NeurIPS*, 2024.

[34] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *ICLR*, 2020.

[35] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *TACL*, 9:53–68, 2021.

[36] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[37] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. FlexPrefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *ICLR*, 2025.

[38] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.

[39] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[40] Zhiyang Chen, Yousong Zhu, Chaoyang Zhao, Guosheng Hu, Wei Zeng, Jinqiao Wang, and Ming Tang. DPT: Deformable patch-based transformer for visual recognition. In *ACMMM*, 2021.

[41] Jifeng Dai, Haozhi Qi, Yuwen Xiong, Yi Li, Guodong Zhang, Han Hu, and Yichen Wei. Deformable convolutional networks. In *ICCV*, 2017.

[42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.

[43] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers. In *ICLR*, 2024.

[44] Robin Strudel, Ricardo Garcia, Ivan Laptev, and Cordelia Schmid. Segmenter: Transformer for semantic segmentation. In *ICCV*, 2021.

[45] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020.

[46] Bowen Cheng, Alex Schwing, and Alexander Kirillov. Per-pixel classification is not all you need for semantic segmentation. In *NeurIPS*, 2021.

[47] Qishuai Wen and Chun-Guang Li. Rethinking decoders for transformer-based semantic segmentation: A compression perspective. In *NeurIPS*, 2024.

[48] Qihang Yu, Mark Weber, Xueqing Deng, Xiaohui Shen, Daniel Cremers, and Liang-Chieh Chen. An image is worth 32 tokens for reconstruction and generation. In *NeurIPS*, 2024.

[49] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *ACL*, 2024.

[50] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[51] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[52] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *NeurIPS*, 2024.

[53] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.

[54] Bolei Zhou, Hang Zhao, Xavier Puig, Tete Xiao, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Semantic understanding of scenes through the ade20k dataset. *IJCV*, 127(3):302–321, 2019.

[55] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena : A benchmark for efficient transformers. In *ICLR*, 2021.

[56] Weikang Meng, Yadan Luo, Xin Li, Dongmei Jiang, and Zheng Zhang. PolaFormer: Polarity-aware linear attention for vision transformers. In *ICLR*, 2025.

[57] Dongchen Han, Xuran Pan, Yizeng Han, Shiji Song, and Gao Huang. FLatten transformer: Vision transformer using focused linear attention. In *ICCV*, 2023.

[58] Feng Wang, Sucheng Ren, Tiezheng Zhang, Predrag Neskovic, Anand Bhattad, Cihang Xie, and Alan Yuille. ViT-5: Vision transformers for the mid-2020s. *arXiv preprint arXiv:2602.08071*, 2026.

[59] Hugo Touvron, Matthieu Cord, and Hervé Jégou. DeiT III: Revenge of the ViT. In *ECCV*, 2022.

[60] Tianlong Chen, Yu Cheng, Zhe Gan, Lu Yuan, Lei Zhang, and Zhangyang Wang. Chasing sparsity in vision transformers: An end-to-end exploration. In *NeurIPS*, 2021.

[61] Cong Wei, Brendan Duke, Ruowei Jiang, Parham Aarabi, Graham W Taylor, and Florian Shkurti. Sparsifiner: Learning sparse instance-dependent attention for efficient vision transformers. In *CVPR*, 2023.

[62] Dongchen Han, Yifan Pu, Zhuofan Xia, Yizeng Han, Xuran Pan, Xiu Li, Jiwen Lu, Shiji Song, and Gao Huang. Bridging the divide: Reconsidering softmax and linear attention. In *NeurIPS*, 2024.

[63] Qishuai Wen, Zhiyuan Huang, and Chun-Guang Li. Towards interpretable and efficient attention: Compressing all by contracting a few. In *NeurIPS*, 2025.

[64] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021.

[65] Andreas Peter Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? Data, augmentation, and regularization in vision transformers. *TMLR*, 2022.

[66] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, David Benjamin Belanger, Lucy J. Colwell, and Adrian Weller. Rethinking attention with performers. In *ICLR*, 2021.

[67] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *AAAI*, 2021.

[68] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. In *ICLR*, 2023.

[69] Liang Zhao, Xiachong Feng, Xiaocheng Feng, Weihong Zhong, Dongliang Xu, Qing Yang, Hongtao Liu, Bing Qin, and Ting Liu. Length extrapolation of transformers: A survey from the perspective of positional encoding. In *EMNLP*, 2024.

Appendix

A Ablation Study

In Tab. 6, we ablate three key design choices in MiTA: 1) the landmark extraction strategy; 2) the number of experts (m) and the number of key-value pairs per expert (k); and 3) the standalone effects of routing and compression. In the following paragraphs, we analyze these results and their implications.

Table 6: Ablation study under the same setting as Tab. 2.

Setting	Acc.	Δ
Landmark Extraction		
Random Selection	70.6	-0.5
Learnable Parameters	66.3	-4.8
1D Average Pooling	70.4	-0.7
2D Average Pooling	71.1	Default
Convolution	67.0	-4.1
Depth-Wise Convolution	68.8	-2.3
Token Merging [68]	**71.3**	**+0.2**
$m \times k$		
16×16	70.0	-1.1
16×25	71.0	-0.1
25×16	70.5	-0.6
25×25	71.1	Default
25×36	71.7	+0.6
36×25	71·2	+0.1
36×36	71.7	+0.6
Compression & Routing		
Compress-and-route	71.1	Default
Compress-only	70.3	-0.8
Route-only	70.8	-0.3

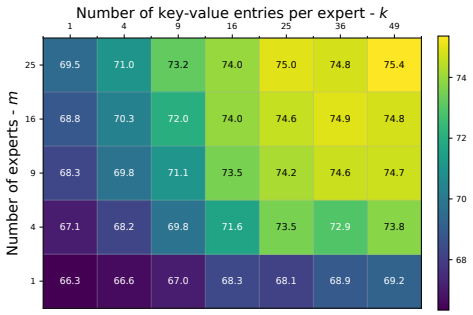


Figure 6: Ablation study of m and k on CIFAR-100. Experiments with other (m, k) were not conducted due to out-of-memory (OOM).

Landmark extraction. Our simple default choice — average pooling over evenly split, non-overlapping rectangular regions — even outperforms parameterized alternatives. This may stem from the fact that landmark queries serve the dual roles of compressed keys and routers, potentially leading to gradient conflicts and suggesting a direction for further improving MiTA. We also observe that more advanced methods, such as ToMe [68] can improve MiTA. However, ToMe must be applied repeatedly to achieve aggressive compression, making it a less lightweight and less elegant solution.

Hyperparameters m and k. Performance improves as either m or k increases, which is expected since MiTA more closely approximates full attention with larger m or k, and recovers full attention when $m = k = N$. More importantly, we observe that performance is more sensitive to k than to m; that is, increasing k is typically more beneficial than increasing m. This trend is further validated in Fig. 6. This finding supports our motivation that gathering top-k activations independently for each

query (e.g., as in Spark Attention [21] and DeepSeek Sparse Attention [22]) is redundant, and that many such activations can instead be shared through routing. For practical use, we provide a simple rule of thumb: first choose a fixed ratio, $\frac{m \times k}{N}$; then start from $m = k$ and explore $k > m$ during subsequent tuning.

Compression and routing. Routing is more critical than compression. Nevertheless, introducing compression further improves performance while also providing notable acceleration, as shown in Tab. 5.

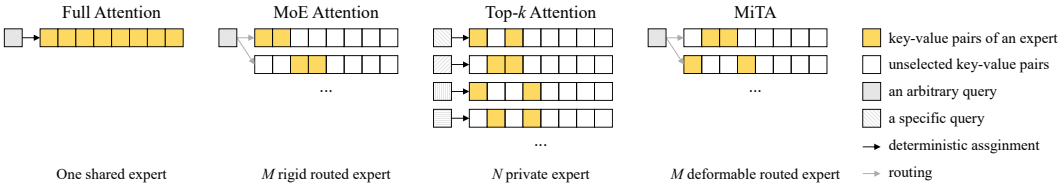


Figure 7: Visual comparison of related attention methods.

B Fit MiTA into the Fast-Weight Scaling Taxonomy

While MiTA is introduced in Sec. 3.2 from the perspective of combining the two complementary scaling strategies, we find that it also enriches the remaining dimensions of the fast-weight scaling taxonomy proposed in Sec. 3.1. This subsection will briefly discuss each of them.

Expert construction. This core dimension determines the expert type and count, and hence the routing topology. An important consideration is what the construction conditions on: the query only (e.g., DAT [19]), the key-value pairs only (e.g., MHLA [13]), or both. When using key-value pairs, the construction may be content-dependent and thus deformable (e.g., DSA [22]) or merely position-driven (e.g., MoBA [9]). While most efficient attention methods condition on one of the above aspects, our MiTA conditions on all of them by extracting landmark queries from queries and then probe the full key–value pool. We illustrate a resulting benefit of this design in Fig. 4.

Expert type and count. Expert types can be categorized in three classes: a) linear layers (e.g., linear attention), b) MLPs (e.g., sparse attention and PVT [15]), c) arbitrary modules (e.g., test-time training). The scaling-by-routing branch of MiTA is restricted to MLP experts, as in sparse attention. However, the scaling-by-compression branch can be implemented through test-time training, thereby generalizing to other modules. As for the expert count, MiTA controls it via the hyperparameter m. One benefit of this design has been discussed in complexity analysis. Another benefit is that it enables expert composition through the routing mechanism.

Routing topology. In MiTA, there is a fixed number of base sparse patterns (i.e., fast-weight experts), which is a substantial improvement over efficient attention with only a single expert, yet is still far fewer than prior top-k attention [21, 22], which has N experts. Crucially, to compensate for this limitation, MiTA can resort to the routing mechanism to compose these m base experts: when routing each query to s experts, the number of effective sparse patterns (i.e., the combinatorial number of experts) is $\binom{m}{s}$. In particular, when $s = 1$, routing may degenerate into clustering, restricting information flow within each cluster. We thus quantify the positional overlap between an expert's gathered key–value pairs and the queries routed to it. As shown in Fig. 8, the overlap remains consistently modest across layers, suggesting that MiTA with $s = 1$ performs routing rather than hard clustering.

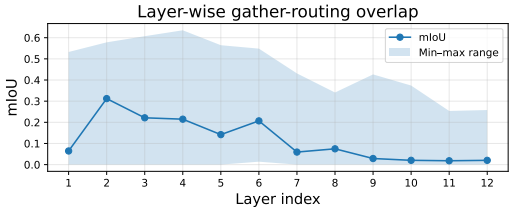
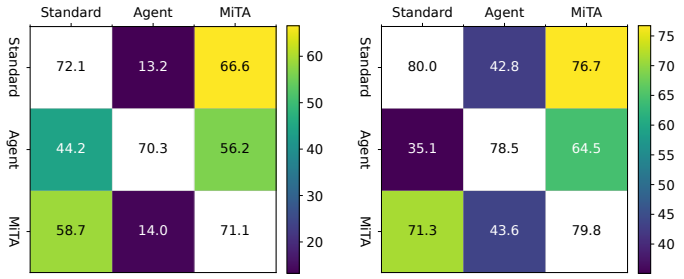


Figure 8: The layer-wise positional overlap (between the key–value pairs gathered by an expert and the queries routed to it) is quantified by mIoU, averaged over experts and attention heads.

Algorithm 1 Mixture-of-Top-k Attention (MiTA)

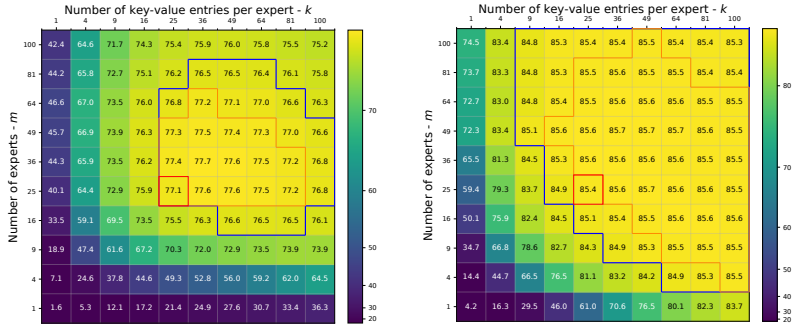
Require: Query, key, value matrices $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{d\times N}$; the number of landmark queries m; the width of each fast-weight expert k; each query is routed to the shared expert and to $s = 1$ additional expert.

// Obtain landmark queries
$\tilde{\boldsymbol{Q}} = \mathrm{AdaptiveAvgPool}(\boldsymbol{Q}, \text{output_size}=m)$ // [d, m]
// Lookup key-value pairs via landmark queries
$\boldsymbol{S}^{\mathrm{kv}} = \boldsymbol{K}^\top \tilde{\boldsymbol{Q}}/\sqrt{d}$ // [N, m]
// Gather the top-k activated key-value pairs
$\mathcal{I}^{\mathrm{kv}} = \mathrm{Flatten}(\mathrm{TopK}(\boldsymbol{S}^{\mathrm{kv}\top}, k, \text{dim}=1))$ // [m*k]
$\boldsymbol{K}^{\mathrm{expt}}, \boldsymbol{V}^{\mathrm{expt}} = \boldsymbol{K}[:, \mathcal{I}^{\mathrm{kv}}], \boldsymbol{V}[:, \mathcal{I}^{\mathrm{kv}}]$ // [d, m*k]
// Obtain landmark values (construct the shared expert)
$\tilde{\boldsymbol{V}} = \boldsymbol{V}\,\mathrm{softmax}(\mathbf{S}^{\mathrm{kv}})$ // [d, m]
// Always route queries to the shared expert
$\boldsymbol{O}^{\mathrm{share}} = \mathrm{FlashAttention}(\boldsymbol{Q}, \tilde{\boldsymbol{Q}}, \tilde{\boldsymbol{V}})$
// Sparsely route queries to other experts
$\mathcal{I}^{\mathrm{expt}} = \mathrm{ArgSort}(\mathrm{ArgMax}(\tilde{\boldsymbol{Q}}^\top \boldsymbol{Q}, \text{dim}=0))$ // [N]
$\boldsymbol{O}^{\mathrm{expt}} = \mathrm{FlashAttention}(\boldsymbol{Q}[:, \mathcal{I}^{\mathrm{expt}}], \boldsymbol{K}^{\mathrm{expt}}, \boldsymbol{V}^{\mathrm{expt}},$
cu_seqlens_q= $\mathrm{CumSum}(\mathrm{BinCount}(\mathcal{I}^{\mathrm{expt}}))$, cu_seqlens_k=$[0, k, 2k, \ldots, mk]$)
// Combine results via online softmax
$\boldsymbol{O} = \mathrm{Combine}(\boldsymbol{O}^{\mathrm{share}}, \boldsymbol{O}^{\mathrm{expt}})$ // [d, N]
return $\boldsymbol{O}$



(a) DeiT-T models (b) DeiT-S models

Figure 9: Evaluation with a different inference attention. The x-axis indexes the training attention, while the y-axis indexes the inference attention. We omit the diagonal entries from the heatmap since they are not of interest. Currently, only standard attention, Agent Attention, and MiTA are included.



(a) MiTA-ViT-T @512 (b) MiTA-ViT-L @224

Figure 10: MiTA's generalization across m and k. The red box marks the training (or finetuning) (m, k) and the corresponding baseline accuracy on ImageNet-1K. The orange boundary marks the inference (m, k) configurations that exceed the baseline accuracy, and the blue boundary marks those achieve 99% of the baseline accuracy. Best viewed when zoomed in.

C Algorithmic Generalization

Broadly, generalization typically refers to performance beyond the training data, e.g., from the training set to the validation set, length extrapolation [69] (from the training resolution to a new one). Motivated by the unifying perspective for efficient attention, this work investigate a new dimension of generalization: *algorithmic generalization*, i.e., performance beyond the training algorithm (e.g., attention mechanism).

In Fig. 10, we study MiTA's generalization across hyperparameters: the expert count m and expert width k. Note that changing an algorithm's hyperparameters is analogous to changing the input resolution. Moreover, in Fig. 9, we study how an attention mechanism generalizes to others. Specifically, given a model trained with one attention mechanism (*training attention*), we replace it with a different mechanism (*inference attention*) at test time. Beyond the generalization setting, where model parameters are fixed, we also finetune models pretrained on a large-scale dataset with a different *finetuning attention* in Tab. 7.

Implementation details. ViTs [65] in Tab. 4 are pretrained on ImageNet-21K with an image size of 224 and a patch size of 16. We finetune them for 50 epochs with the AdamW optimizer, using the listed attention mechanisms (including standard attention). The model in Fig. 10(a) is similarly finetuned, but starts from a ViT pretrained with an image size of 384. Models in Fig. 9 are the same as those in Tab. 2.

Results. Fig. 10 suggests that MiTA generalizes well when scaling up the expert count and width: many configurations exceeds the original result. This indicates a promising strategy: train MiTA with smaller m and k for efficiency, then increase m or k at inference to obtain gains. In Fig. 9, we observe that standard attention and MiTA generalize to each other better than other pairs, especially when training with standard attention and testing with MiTA, which retains over 95% of the original performance with linear complexity. Regarding finetuning, Tab. 7 shows that parameters pretrained with standard attention transfer more easily to MiTA, while Agent Attention struggles to narrow this gap even when increasing the number of agent tokens.

Table 7: Finetuning results. We finetune ImageNet-21K pretrained ViTs [65] on ImageNet-1K, replacing the standard attention with Agent Attention or MiTA Attention. †: the number of agent tokens (i.e., landmark queries) is increased from 49 (default) to 64.

Methods	Tiny	Small	Base	Large
Standard Attention	76.9	81.2	84.4	85.9
Linear Attention	73.2	78.6	79.8	80.7
Agent Attention	74.6	79.7	81.4	83.5
Agent Attention†	74.7	79.9	81.5	83.8
MiTA Attention	75.6 (-1.3)	80.9 (-0.3)	82.8 (-1.6)	85.5 (-0.4)

D Limitations

This work primarily focuses on the vision domain, and we do not evaluate the proposed method in the decoding phase of LLMs. This choice is mainly due to our limited computational resources. That said, the method and related work are presented in a modality-agnostic manner, and we discuss their applicability beyond vision. We consider the application to LLMs as an important direction for future work. The main contribution of this paper is conceptual, namely a unified perspective that facilitates principled improvements, rather than a domain-specific instantiation.